LEARN BEING FORWARD

Programming — JavaScript Programming

Working with APIs & JSON

COMPLETE LEARNING GUIDE

What You'll Learn

Nearly every modern application talks to a server at some point, and JSON (JavaScript Object Notation) is the universal data format that exchange happens in — a lightweight, language-agnostic text format that happens to map directly onto JavaScript's own object and array literal syntax. This sub-module covers JSON itself and its conversion methods, then walks through the Fetch API end to end: making GET requests, the easy-to-miss detail that fetch does not reject on HTTP error statuses, sending data with POST/PUT, setting the right headers, shaping the response data you receive, and cancelling an in-flight request with AbortController — the complete, practical toolkit for a JavaScript program (browser or Node) to consume any REST API correctly.

1. What Is JSON and Why APIs Use It

JSON is a plain-text data format — objects as key-value pairs in curly braces, arrays in square brackets, and only a handful of value types (string, number, boolean, null, object, array) — deliberately simpler than a full JavaScript object literal, which is why JSON keys must always be double-quoted strings and JSON cannot contain functions, undefined, or comments. Its appeal as an API exchange format is its combination of being genuinely human-readable and natively parseable in virtually every programming language, not just JavaScript, making it the de facto standard for REST APIs across the entire industry.

JAVASCRIPT

```
// Valid JSON (as a string, exactly as it travels over the network):
// { "name": "Asha", "active": true, "tags": ["admin", "beta"] }

// This is NOT valid JSON (unquoted key, trailing comma, a function):
// { name: "Asha", greet: function() {}, }
```

2. JSON.stringify and JSON.parse

JSON.stringify(value) converts a JavaScript value into its JSON text representation — the form actually sent over the network in a request body. JSON.parse(text) does the reverse, converting a JSON string back into a real JavaScript object or array you can work with directly. Both operations only round-trip the types JSON actually supports: functions and undefined properties are silently dropped by stringify, and Date objects get converted to plain ISO strings (and stay strings after parsing — they don't automatically become Date objects again).

```javascript
const user = { name: "Raj", age: 28, login: () => {}, lastSeen: undefined };
const json = JSON.stringify(user);
console.log(json); // '{"name":"Raj","age":28}' — login and lastSeen were
silently dropped

const parsed = JSON.parse(json);
console.log(parsed.age); // 28 — now a real JS object again
```

3. Fetching Data with the Fetch API

fetch(url) sends an HTTP GET request by default and returns a Promise that resolves with a Response object representing the server's reply. The response body isn't parsed automatically — calling response.json() (or .text(), or .blob()) returns its own separate Promise that must also be awaited before you get the actual parsed data. This two-step shape (await the fetch, then await the body-parsing method) is the standard pattern for essentially every fetch call.

```javascript
async function getUser(id) {
  const response = await fetch(`https://api.example.com/users/${id}`);
  const user = await response.json(); // parsing the body is ALSO async
  return user;
}
```

4. Checking response.ok and Handling HTTP Errors

fetch's returned Promise only rejects on genuine network-level failures — DNS failure, no connection, a CORS block — never because the server responded with an error status like 404 or 500; as far as fetch itself is concerned, any response from the server at all counts as success. This means response.ok (true only for status codes in the 200-299 range) or response.status must be checked explicitly inside the then/await code; skipping that check means a server's 404 error page gets silently treated as if it were valid data.

```javascript
async function getUser(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }
  return response.json();
}
```

5. Sending Data: POST and PUT Requests

fetch's default method is GET; sending data requires an options object as the second argument specifying method: 'POST' (or 'PUT' for a full update, 'PATCH' for a partial one) plus a body containing the payload. The body must be a string (or other supported type like FormData) — a plain JavaScript object passed directly as the body is not automatically converted, so JSON.stringify() must be called on it explicitly before sending.

```javascript
async function createUser(newUser) {
  const response = await fetch("/api/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(newUser), // must stringify — fetch won't do it for you
  });
  if (!response.ok) throw new Error(`Create failed: ${response.status}`);
  return response.json();
}
```

6. Headers and Content-Type

HTTP headers carry metadata about a request or response separately from the actual body content. The Content-Type request header tells the server how to interpret the body you're sending — application/json for a JSON payload — and omitting it (or sending it incorrectly) can cause a server to misinterpret or outright reject an otherwise well-formed request body. Response headers can be read from fetch's Response object via response.headers.get('header-name'), useful for things like reading a server-provided pagination total or rate-limit count.

```javascript
const response = await fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${token}`,
  },
  body: JSON.stringify({ name: "Priya" }),
});
console.log(response.headers.get("X-Total-Count"));
```

7. Shaping and Validating the Response You Receive

Real APIs rarely return data in exactly the shape your UI or logic wants directly — destructuring the parsed response immediately after awaiting it (const { data, meta } = await response.json()) keeps the rest of the code working with only the fields it actually needs. Since the server's response is inherently untrusted input from the program's own perspective, checking that expected fields actually exist (and are the expected type) before using them — rather than assuming the shape is always exactly as documented — prevents a malformed or unexpected response from causing confusing downstream errors.

```javascript
async function getActiveUserNames() {
  const response = await fetch("/api/users");
  if (!response.ok) throw new Error(`Status ${response.status}`);

  const { data } = await response.json();
  if (!Array.isArray(data)) throw new Error("Unexpected response shape");

  return data.filter((u) => u.active).map((u) => u.name);
}
```

8. Cancelling Requests with AbortController

A fetch request left running after the code that triggered it no longer cares about the result — a user typed a newer search query, or navigated away — wastes bandwidth and risks a stale response overwriting fresher data if it resolves after a newer request. AbortController creates a signal that can be passed to fetch's options and triggered later via controller.abort(), which causes the in-flight fetch Promise to reject with an AbortError — a rejection that should typically be recognized and ignored, since it's an intentional cancellation rather than a genuine failure.

```javascript
let controller;

async function search(query) {
  controller?.abort(); // cancel any still-running previous search
  controller = new AbortController();

  try {
    const response = await fetch(`/api/search?q=${query}`, { signal:
controller.signal });
    return response.json();
  } catch (err) {
    if (err.name === "AbortError") return; // expected — a newer search
superseded this one
    throw err;
  }
}
```

Common Pitfalls

- Assuming fetch rejects on a 404 or 500 response — it only rejects on network-level failures, so response.ok must always be checked explicitly.
- Forgetting that response.json() itself returns a Promise and must be awaited before the parsed data is usable.
- Passing a plain JavaScript object directly as a fetch body instead of running it through JSON.stringify() first.

- Sending a POST/PUT request without setting the Content-Type: application/json header, risking the server misinterpreting or rejecting the payload.
- Calling JSON.parse() on a value that isn't a valid JSON string (or is already a parsed object) without catching the resulting SyntaxError.
- Trusting an API response's shape blindly without checking that expected fields actually exist before accessing them.
- Leaving a slow, now-irrelevant fetch request running instead of cancelling it with AbortController when newer input has superseded it.

Key Takeaways

- JSON only supports strings, numbers, booleans, null, objects, and arrays — functions and undefined are silently dropped by JSON.stringify.
- fetch only rejects on network failures; always check response.ok (or response.status) to catch HTTP-level errors like 404 or 500.
- Parsing a fetch response body (response.json()) is itself asynchronous and must be awaited separately from the fetch call.
- A request body must be a string — call JSON.stringify() on any object before sending it, and set the Content-Type header to match.
- Validate the shape of data you receive from an API rather than assuming it always matches the documentation exactly.
- AbortController cancels an in-flight fetch cleanly, which is the right tool whenever a newer request supersedes an older, still-pending one.

Learn More

MDN — Using the Fetch API
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN — JSON
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON

MDN — AbortController
https://developer.mozilla.org/en-US/docs/Web/API/AbortController